

Aula Detalhada - Formatos, Tipos De Instrução E Modos De Endereçamento

Tema do dia: formato de instrução, opcode, campos de registradores, imediato, endereço, tipos de instrução, instruções aritméticas/lógicas, acesso à memória, desvios, modos de endereçamento e leitura de instruções em assembly

Aula na sequência: 26

Objetivo: entender como uma instrução é organizada em campos, quais tipos de instrução aparecem em uma arquitetura e como os modos de endereçamento indicam onde estão os operandos.

1. Onde Esta Aula Entra No Estudo?

Na Aula 25, você viu a ponte:

```
algoritmo  
-> assembly  
-> instrução de máquina  
-> hardware
```

Agora vamos abrir a instrução por dentro.

Na aula anterior, aparecia algo assim:

```
ADD R1, R2, R3
```

Leitura:

```
R1 = R2 + R3
```

Mas para o processador executar isso, a instrução precisa virar bits.

Esses bits precisam dizer:

```
qual operação fazer?  
quais registradores usar?  
existe constante imediata?  
existe endereço de memória?  
é uma instrução de desvio?
```

Essa aula responde exatamente isso.

2. Ideia Central

Uma instrução é um comando entendido pelo processador.

Em assembly, ela é escrita de forma legível:

```
ADD R1, R2, R3
```

Em código de máquina, ela vira uma sequência de bits.

Exemplo conceitual:

```
opcode | rd | rs1 | rs2
```

Onde:

```
opcode -> operação  
rd      -> registrador destino  
rs1     -> registrador fonte 1  
rs2     -> registrador fonte 2
```

Então:

```
instrução = opcode + campos de operandos/controle
```

O processador lê esses campos e configura o datapath.

3. O Que É Opcode?

Opcode significa:

```
operation code
```

Ou:

```
código da operação
```

É o campo da instrução que diz o que a CPU deve fazer.

Exemplos:

```
ADD  -> somar
SUB  -> subtrair
AND  -> AND bit a bit
OR   -> OR bit a bit
LOAD -> ler memória
STORE -> escrever memória
BEQ  -> desviar se igual
JUMP -> salto incondicional
```

Em máquina real, cada operação tem um código binário.

Exemplo didático:

```
0000 -> ADD
0001 -> SUB
0010 -> AND
0011 -> OR
0100 -> LOAD
0101 -> STORE
0110 -> BEQ
0111 -> JUMP
```

Não precisa decorar códigos de uma arquitetura específica.

Precisa entender:

```
opcode identifica a operação
```

4. Campos Comuns De Uma Instrução

Uma instrução pode ter campos como:

```
opcode
registrador destino
registrador fonte
imediato
endereço
deslocamento
código de função
```

4.1 Registrador destino

É onde o resultado será gravado.

Exemplo:

```
ADD R1, R2, R3
```

Destino:

```
R1
```

4.2 Registradores fonte

São os registradores lidos como entrada.

Exemplo:

```
ADD R1, R2, R3
```

Fontes:

```
R2  
R3
```

4.3 Imediato

Imediato é uma constante dentro da própria instrução.

Exemplo:

```
ADDI R1, R2, 5
```

Leitura:

```
R1 = R2 + 5
```

O **5** não vem de registrador nem da memória.

Ele está codificado na instrução.

4.4 Endereço ou deslocamento

Instruções de memória e desvio podem carregar um campo de endereço ou deslocamento.

Exemplo:

```
LOAD R1, [R2 + 8]
```

Aqui:

```
R2 -> registrador base  
8  -> deslocamento
```

Endereço efetivo:

```
R2 + 8
```

5. Formato De Instrução

Formato de instrução é o jeito como os bits da instrução são divididos.

Exemplo didático de 16 bits:

```
bits:   15..12  11..8  7..4  3..0  
campo: opcode   rd     rs1   rs2
```

Isso significa:

```
4 bits para opcode  
4 bits para registrador destino  
4 bits para fonte 1  
4 bits para fonte 2
```

Se o opcode tem 4 bits, pode representar:

```
2^4 = 16 operações diferentes
```

Se o campo de registrador tem 4 bits, pode selecionar:

```
2^4 = 16 registradores
```

Essa lógica de 2^n volta muito.

6. Por Que Existem Vários Formatos?

Nem toda instrução precisa dos mesmos campos.

Compare:

```
ADD R1, R2, R3
```

Precisa de:

```
opcode  
rd  
rs1  
rs2
```

Agora:

```
ADDI R1, R2, 10
```

Precisa de:

```
opcode  
rd  
rs1  
imediato
```

Agora:

```
JUMP alvo
```

Precisa de:

```
opcode  
endereço ou deslocamento
```

Por isso arquiteturas costumam ter formatos diferentes.

7. Formato Tipo R

Formato tipo R é usado para operações entre registradores.

Pense em **R** como:

register

Exemplos:

```
ADD R1, R2, R3
SUB R4, R5, R6
AND R7, R8, R9
OR  R1, R1, R2
```

Estrutura conceitual:

opcode | rd | rs1 | rs2

Leitura:

rd = rs1 operação rs2

Exemplo:

```
ADD R1, R2, R3
```

Campos:

```
opcode = ADD
rd      = R1
rs1     = R2
rs2     = R3
```

Resultado:

R1 = R2 + R3

8. Formato Tipo I

Formato tipo I é usado para instruções com imediato.

Pense em **I** como:

immediate

Exemplos:

```
ADDI R1, R2, 5
ANDI R3, R4, 15
LOAD R1, [R2 + 8]
```

Estrutura conceitual:

```
opcode | rd | rs1 | imediato
```

No caso de memória:

```
opcode | registrador dado | registrador base | deslocamento
```

Exemplo:

```
ADDI R1, R2, 5
```

Campos:

```
opcode    = ADDI
rd         = R1
rs1        = R2
imediato   = 5
```

Resultado:

```
R1 = R2 + 5
```

9. Formato Tipo J Ou Desvio

Formato tipo J é usado para saltos.

Pense em **J** como:

```
jump
```

Exemplo:

```
JUMP alvo
```

Estrutura conceitual:

```
opcode | endereço
```

Ou:

```
opcode | deslocamento
```

Depende da arquitetura.

Em vez de produzir resultado na ULA para salvar em registrador, o objetivo é mudar o fluxo:

```
PC = novo endereço
```

PC significa:

```
program counter
```

É o registrador que guarda o endereço da próxima instrução.

10. Instruções De 3, 2, 1 E 0 Endereços

Esse é um jeito clássico de classificar instruções pelo número de operandos explícitos.

10.1 Instrução de 3 endereços

Exemplo:

```
ADD R1, R2, R3
```

Leitura:

```
R1 = R2 + R3
```

Três endereços/operandos:

```
destino  
fonte 1  
fonte 2
```

Vantagem:

clara
não destrói operandos fonte

10.2 Instrução de 2 endereços

Exemplo:

ADD R1, R2

Leitura comum:

$R1 = R1 + R2$

Um operando também é destino.

Vantagem:

instrução menor

Desvantagem:

sobrescreve um dos operandos

10.3 Instrução de 1 endereço

Exemplo conceitual com acumulador:

ADD X

Leitura:

$ACC = ACC + M[X]$

ACC é acumulador.

Um operando fica implícito.

10.4 Instrução de 0 endereços

Usada em máquinas de pilha.

Exemplo:

ADD

Leitura:

retira dois valores do topo da pilha
soma
empilha o resultado

Resumo:

Tipo	Exemplo	Ideia
3 endereços	ADD R1,R2,R3	destino separado das fontes
2 endereços	ADD R1,R2	destino também é fonte
1 endereço	ADD X	usa acumulador implícito
0 endereços	ADD	usa pilha

11. Tipos De Instrução

Uma ISA normalmente possui grupos de instruções.

ISA significa:

Instruction Set Architecture

Ou:

arquitetura do conjunto de instruções

Principais grupos:

aritméticas
lógicas
deslocamento
transferência de dados
controle de fluxo
comparação/teste
entrada/saída, em algumas arquiteturas

12. Instruções Aritméticas

Executam operações numéricas.

Exemplos:

```
ADD
SUB
MUL
DIV
ADDI
```

Exemplo:

```
ADD R1, R2, R3
```

Leitura:

```
R1 = R2 + R3
```

Exemplo com imediato:

```
ADDI R1, R1, 1
```

Leitura:

```
R1 = R1 + 1
```

Isso é muito comum para incrementar contadores.

13. Instruções Lógicas

Executam operações bit a bit.

Exemplos:

```
AND
OR
XOR
NOT
ANDI
ORI
```

Exemplo:

```
AND R1, R2, R3
```

Leitura:

```
R1 = R2 AND R3
```

Operação bit a bit:

```
R2 = 1100  
R3 = 1010  
AND = 1000
```

Então:

```
R1 = 1000
```

Usos comuns:

```
máscaras de bits  
limpar bits  
testar bits  
combinar flags
```

14. Instruções De Deslocamento

Deslocam bits para esquerda ou direita.

Exemplos:

```
SHL  
SHR  
SAR  
SLL  
SRL
```

Nomes variam por arquitetura.

Ideias:

shift left -> desloca para esquerda
shift right lógico -> desloca para direita inserindo 0
shift right aritmético -> preserva bit de sinal

Exemplo:

$0011 \ll 1 = 0110$

Isso equivale a:

$3 * 2 = 6$

Mas só quando não ocorre overflow e a interpretação permite.

15. Instruções De Transferência De Dados

Movem dados entre registradores e memória.

Exemplos:

LOAD
STORE
MOV
PUSH
POP

15.1 LOAD

Carrega dado da memória para registrador.

LOAD R1, [R2 + 8]

Leitura:

$R1 = \text{Mem}[R2 + 8]$

15.2 STORE

Grava dado de registrador na memória.

```
STORE [R2 + 8], R1
```

Leitura:

```
Mem[R2 + 8] = R1
```

Pegadinha:

```
LOAD lê memória  
STORE escreve memória
```

16. Instruções De Controle De Fluxo

Alteram a próxima instrução executada.

Exemplos:

```
JUMP  
BEQ  
BNE  
BLT  
BGT  
CALL  
RET
```

16.1 Jump

Salto incondicional.

```
JUMP loop
```

Leitura:

```
vá para loop
```

16.2 Branch condicional

Desvia apenas se uma condição for satisfeita.

Exemplo:

```
BEQ R1, R2, alvo
```

Leitura:

```
se R1 == R2, vá para alvo
```

Se a condição for falsa:

```
continua na próxima instrução
```

17. Comparação E Flags

Algumas arquiteturas possuem instrução `CMP`.

Exemplo:

```
CMP R1, R2
```

Ela pode fazer uma subtração interna:

```
R1 - R2
```

Mas sem guardar o resultado em registrador.

Ela atualiza flags:

```
Z -> zero  
N -> negativo  
C -> carry  
V -> overflow
```

Depois, uma instrução de desvio usa essas flags.

Exemplo:

```
CMP R1, R2  
BEQ igual
```

Leitura:

```
compare R1 e R2  
se forem iguais, desvie para igual
```

Em arquiteturas RISC, a comparação pode aparecer direto no branch:

```
BEQ R1, R2, igual
```

Ou seja:

```
compare e desvie na mesma instrução
```

18. Load-Store Vs Memória-Memória

Algumas arquiteturas são do tipo load-store.

Nelas, operações aritméticas usam registradores.

Exemplo:

```
LOAD R1, [A]  
LOAD R2, [B]  
ADD R3, R1, R2  
STORE [C], R3
```

Ideia:

```
memória só é acessada por LOAD/STORE  
ULA trabalha com registradores
```

Outras arquiteturas permitem operações diretamente com memória.

Exemplo conceitual:

```
ADD [C], [A], [B]
```

Mas isso costuma exigir instruções mais complexas.

Resumo:

```
load-store -> simples, comum em RISC  
memória-memória -> mais complexa, comum em ideias CISC antigas/conceituais
```

Para prova introdutória:

```
registradores são rápidos  
memória é mais lenta
```

19. Modos De Endereçamento

Modo de endereçamento é a forma como a instrução indica onde está o operando.

Pergunta central:

o dado está onde?

Possíveis respostas:

dentro da instrução
em um registrador
em um endereço fixo de memória
em um endereço guardado em registrador
em base + deslocamento
perto do PC
no topo da pilha

20. Endereçamento Imediato

O operando está dentro da própria instrução.

Exemplo:

```
ADDI R1, R2, 10
```

O `10` é imediato.

Leitura:

```
R1 = R2 + 10
```

Vantagem:

não precisa buscar o operando na memória

Limitação:

o valor precisa caber no campo imediato

Exemplo de prova:

Se o imediato tem 8 bits sem sinal, ele representa:

0 até 255

Se o imediato tem 8 bits com sinal em complemento de 2:

-128 até 127

21. Endereçamento Por Registrador

O operando está em um registrador.

Exemplo:

ADD R1, R2, R3

Operandos:

R2

R3

Resultado:

R1

Vantagem:

rápido

campo pequeno

Por quê?

Se há 32 registradores, bastam:

5 bits

Porque:

$2^5 = 32$

22. Endereçamento Direto Ou Absoluto

A instrução contém diretamente o endereço de memória.

Exemplo:

```
LOAD R1, [1000]
```

Leitura:

```
R1 = Mem[1000]
```

O endereço efetivo é:

```
1000
```

Vantagem:

```
simples de entender
```

Limitação:

```
campo de endereço pode precisar ser grande
```

23. Endereçamento Indireto Por Registrador

O registrador guarda o endereço da memória.

Exemplo:

```
LOAD R1, [R2]
```

Leitura:

```
R1 = Mem[R2]
```

Se:

```
R2 = 1000  
Mem[1000] = 55
```

Então:

```
R1 = 55
```

Pegadinha:

```
R2 não é o dado  
R2 é o endereço do dado
```

24. Endereçamento Base + Deslocamento

Também chamado de:

```
base + offset  
base + deslocamento  
base displacement
```

Exemplo:

```
LOAD R1, [R2 + 8]
```

Endereço efetivo:

```
R2 + 8
```

Se:

```
R2 = 1000
```

Então:

```
endereço efetivo = 1008
```

Leitura:

```
R1 = Mem[1008]
```

Uso comum:

acessar campos de estruturas
acessar variáveis locais na pilha
acessar elementos próximos de um endereço base

25. Endereçamento Indexado

Usa um registrador como índice.

Exemplo conceitual:

```
LOAD R1, [BASE + Ri]
```

Ou:

```
LOAD R1, [Rbase + Rindex]
```

Ideia:

```
endereço efetivo = base + índice
```

Muito usado para vetores.

Exemplo:

```
vetor[i]
```

Pode virar:

```
endereço = base_do_vetor + i * tamanho_do_elemento
```

Se cada elemento tem 4 bytes:

```
endereço = base + 4.i
```

Algumas arquiteturas permitem escala no modo de endereçamento.

Exemplo conceitual:

```
[Rbase + Rindex*4]
```

26. Endereçamento Relativo Ao PC

PC é o registrador da próxima instrução.

Endereçamento relativo ao PC usa:

```
PC + deslocamento
```

Muito comum em branches.

Exemplo:

```
BEQ R1, R2, +12
```

Leitura:

```
se R1 == R2, PC = PC + 12
```

Vantagem:

```
código pode ser deslocado na memória com mais facilidade
```

Também economiza bits:

```
em vez de guardar endereço completo,  
guarda apenas distância até o alvo
```

27. Endereçamento Implícito

O operando não aparece explicitamente.

Exemplo:

```
CLR C
```

Pode limpar uma flag de carry, dependendo da arquitetura.

Outro exemplo:

```
RET
```

Pode usar implicitamente:

endereço salvo na pilha

Em máquinas com acumulador:

ADD X

pode significar:

$ACC = ACC + Mem[X]$

O acumulador está implícito.

28. Endereçamento Por Pilha

Usa o topo da pilha como local implícito de operandos.

Exemplo:

```
PUSH R1
PUSH R2
ADD
POP R3
```

Leitura:

```
empilha R1
empilha R2
ADD soma os dois valores do topo
POP salva resultado em R3
```

Muito usado em:

```
chamadas de função
retornos
salvar registradores
variáveis locais
```

29. Como Calcular Bits De Campos

Essa parte cai bastante.

29.1 Bits para registradores

Se existem N registradores, precisa de:

$\log_2(N)$ bits

Exemplos:

8 registradores -> 3 bits
16 registradores -> 4 bits
32 registradores -> 5 bits
64 registradores -> 6 bits

29.2 Bits para opcode

Se há M operações diferentes, precisa de bits suficientes para representar M códigos.

Exemplos:

8 operações -> 3 bits
16 operações -> 4 bits
20 operações -> 5 bits, pois 4 bits só dão 16 códigos

29.3 Bits para imediato

Se o imediato tem k bits sem sinal:

0 até $2^k - 1$

Se o imediato tem k bits com sinal em complemento de 2:

$-2^{(k-1)}$ até $2^{(k-1)}-1$

Exemplo:

imediato de 12 bits com sinal

Intervalo:

-2048 até 2047

30. Exemplo Completo De Formato

Suponha uma instrução de 16 bits:

```
opcode | rd | rs1 | rs2
4 bits | 4b | 4b | 4b
```

Perguntas:

1. Quantas operações diferentes cabem no opcode?
2. Quantos registradores podem ser endereçados?
3. Esse formato serve bem para imediato?

Respostas:

1. Opcode de 4 bits:

```
2^4 = 16 operações
```

1. Campo de registrador de 4 bits:

```
2^4 = 16 registradores
```

1. Não diretamente, porque os campos estão reservados para registradores.

Para imediato, seria necessário outro formato:

```
opcode | rd | rs1 | imediato
4 bits | 4b | 4b | 4b
```

Nesse caso, o imediato teria apenas 4 bits.

31. Exemplo De Rastreio

Considere:

```
R1 = 10
R2 = 20
Mem[108] = 7
```

Instruções:

```
ADDI R3, R1, 5
LOAD R4, [R1 + 98]
ADD R5, R3, R4
```

Passo 1:

```
ADDI R3, R1, 5
R3 = 10 + 5 = 15
```

Passo 2:

```
LOAD R4, [R1 + 98]
endereço = 10 + 98 = 108
R4 = Mem[108] = 7
```

Passo 3:

```
ADD R5, R3, R4
R5 = 15 + 7 = 22
```

Resultado final:

```
R3 = 15
R4 = 7
R5 = 22
```

32. Como Questões Costumam Cobrar

As questões podem pedir:

```
identificar opcode
identificar registrador destino
identificar registradores fonte
calcular quantidade de operações possíveis
calcular quantidade de registradores
identificar modo de endereçamento
calcular endereço efetivo
rastrear valores de registradores
diferenciar LOAD de STORE
diferenciar branch de jump
```

Exemplo de enunciado:

Em `LOAD R1, [R2 + 12]`, qual é o modo de endereçamento?

Resposta:

base + deslocamento

Outro:

Se $R2=100$, qual endereço é acessado?

Resposta:

112

33. Erros Comuns

33.1 Confundir imediato com endereço

Em:

`ADDI R1, R2, 10`

10 é valor.

Em:

`LOAD R1, [10]`

10 é endereço.

33.2 Confundir registrador com memória

Em:

`ADD R1, R2, R3`

R2 é o valor no registrador.

Em:

```
LOAD R1, [R2]
```

R2 é endereço.

33.3 Confundir destino em STORE

Em:

```
STORE [R2 + 8], R1
```

O destino é a memória.

O valor vem de:

```
R1
```

33.4 Achar que toda instrução acessa memória

Não.

Em arquitetura load-store, normalmente:

```
ADD, SUB, AND -> registradores  
LOAD, STORE -> memória
```

33.5 Esquecer que branch muda o PC

Branch e jump não salvam resultado em registrador comum.

Eles alteram:

```
PC
```

34. Exercícios

34.1 Conceituais

1. O que é opcode?
1. Qual é a diferença entre registrador fonte e registrador destino?
1. O que é imediato?

1. O que é formato de instrução?
1. Por que uma arquitetura pode ter mais de um formato de instrução?
1. O que caracteriza uma instrução tipo R?
1. O que caracteriza uma instrução tipo I?
1. O que faz uma instrução de desvio?
1. Qual é a diferença entre `LOAD` e `STORE` ?
1. O que é modo de endereçamento?
1. Qual é a diferença entre endereçamento imediato e direto?
1. Qual é a diferença entre endereçamento por registrador e indireto por registrador?
1. O que é endereçamento base + deslocamento?
1. Por que PC-relative é útil em branches?
1. O que significa uma arquitetura load-store?

34.2 Cálculos

1. Um campo de opcode tem 5 bits. Quantas operações diferentes ele pode codificar?
1. Uma CPU tem 32 registradores. Quantos bits são necessários para selecionar um registrador?
1. Uma CPU tem 20 registradores. Quantos bits são necessários no campo de registrador?
1. Um imediato sem sinal tem 8 bits. Qual o intervalo de valores?
1. Um imediato com sinal em complemento de 2 tem 8 bits. Qual o intervalo?
1. Um formato tem 16 bits: `opcode` de 4 bits e três campos de registrador do mesmo tamanho. Quantos bits sobram para cada registrador?
1. Um formato de 32 bits usa `opcode` de 6 bits, `rd` de 5 bits e `rs1` de 5 bits. Quantos bits sobram para imediato?
1. Em `LOAD R1, [R2 + 12]`, se `R2=1000`, qual endereço é acessado?
1. Em `BEQ R1, R2, +16`, se a condição for verdadeira, o PC vai para onde em termos conceituais?
1. Uma LUT de Aula 24 tem 5 entradas. Quantas posições ela tem? Por que essa ideia lembra campo de endereço?

34.3 Múltipla escolha

1. Em `ADDI R1, R2, 7`, o valor `7` é:
A) opcode.

B) registrador destino.

C) imediato.

D) endereço absoluto obrigatoriamente.

1. Em `LOAD R3, [R4]`, o modo de endereçamento é:

A) imediato.

B) por registrador para operação aritmética.

C) indireto por registrador.

D) implícito.

1. Em `LOAD R1, [R2 + 8]`, o endereço efetivo é:

A) `R1 + 8`.

B) `R2 + 8`.

C) `R1 + R2`.

D) `8`.

1. Uma instrução tipo R normalmente usa:

A) apenas opcode e endereço longo.

B) registradores como operandos.

C) apenas pilha.

D) apenas imediato.

1. Em arquitetura load-store:

A) toda instrução acessa memória.

B) a ULA opera diretamente sobre dois endereços de memória.

C) LOAD/STORE acessam memória e operações aritméticas usam registradores.

D) não existem registradores.

1. Uma instrução de branch altera principalmente:

A) o PC.

B) a tensão de alimentação.

C) a quantidade de registradores.

D) o tamanho da memória.

1. Se um campo tem `n` bits, ele pode representar:

A) `n` valores.

B) `2n` valores.

C) `2^n` valores.

D) `n^2` valores.

34.4 Rastreio

Considere:

```
R1 = 4  
R2 = 10  
Mem[14] = 99
```

Instruções:

```
ADD R3, R1, R2  
LOAD R4, [R3]  
ADDI R5, R4, 1
```

1. Qual o valor final de R3 ?
 1. Qual o valor final de R4 ?
 1. Qual o valor final de R5 ?
-

35. Gabarito

1. É o código da operação, o campo que diz qual ação a CPU deve executar.
1. Fonte é lido como entrada; destino recebe o resultado.
1. É uma constante codificada dentro da própria instrução.
1. É a divisão dos bits da instrução em campos como opcode, registradores, imediato e endereço.
1. Porque instruções diferentes precisam de campos diferentes: algumas usam registradores, outras imediato, outras endereço/desvio.
1. Usa registradores como operandos principais, normalmente `opcode | rd | rs1 | rs2`.
1. Usa campo imediato, normalmente `opcode | rd | rs1 | imediato`.
1. Altera o fluxo de execução, mudando o PC quando a condição ou salto ocorre.
1. `LOAD` lê da memória para registrador; `STORE` escreve de registrador para memória.
1. É a forma como a instrução indica onde está o operando.
1. Imediato é valor dentro da instrução; direto é endereço de memória dentro da instrução.
1. Por registrador usa o valor do registrador como operando; indireto usa o registrador como endereço de memória.
1. O endereço efetivo é calculado como registrador base + constante/deslocamento.

1. Porque permite representar o alvo como distância em relação ao PC, economizando bits e facilitando código relocável.
 1. Significa que a memória é acessada por LOAD/STORE, enquanto operações aritméticas/lógicas usam registradores.
 1. $2^5 = 32$ operações.
 1. 5 bits, pois $2^5 = 32$.
 1. 5 bits, pois 4 bits dão só 16 e 5 bits dão 32.
 1. 0 até 255.
 1. -128 até 127.
 1. Sobram 12 bits para três campos, então 4 bits por registrador.
 1. $32 - 6 - 5 - 5 = 16$ bits.
 1. $1000 + 12 = 1012$.
 1. Conceitualmente, $PC = PC + 16$, se a arquitetura usar esse deslocamento diretamente.
 1. $2^5 = 32$ posições. Lembra campo de endereço porque as entradas selecionam uma das posições.
 1. C.
 1. C.
 1. B.
 1. B.
 1. C.
 1. A.
 1. C.
 1. $R3 = R1 + R2 = 4 + 10 = 14$.
 1. $R4 = \text{Mem}[R3] = \text{Mem}[14] = 99$.
 1. $R5 = R4 + 1 = 99 + 1 = 100$.
-

36. O Que Memorizar

Memorize:

opcode -> operação
rd -> destino
rs -> fonte
imm -> constante dentro da instrução

Memorize os formatos:

tipo R -> registradores
tipo I -> imediato ou deslocamento
tipo J -> salto/desvio

Memorize os modos:

imediato -> valor na instrução
registrador -> valor no registrador
direto/absoluto -> endereço na instrução
indireto por registrador -> registrador guarda endereço
base + deslocamento -> endereço = base + offset
PC-relative -> endereço = PC + offset
implícito -> operando subentendido
pilha -> topo da pilha

Memorize os cálculos:

n bits -> 2^n códigos
32 registradores -> 5 bits
imediato com sinal de k bits -> $-2^{(k-1)}$ até $2^{(k-1)}-1$

Memorize a pegadinha:

LOAD R1, [R2] usa R2 como endereço
ADD R1, R2, R3 usa R2 como valor

37. Plano De Estudo Para Esta Aula

Tempo sugerido: **45 a 70 minutos.**

37.1 Primeira passada

Leia com foco em:

```
opcode
registradores fonte/destino
imediato
LOAD/STORE
branch/jump
modos de endereçamento
```

37.2 Segunda passada

Refaça sem olhar:

```
tipo R, tipo I, tipo J
diferença entre imediato e direto
diferença entre registrador e indireto por registrador
base + deslocamento
PC-relative
```

37.3 Exercícios prioritários

Se tiver pouco tempo, faça:

```
16, 17, 18, 19, 20, 22, 23, 26, 27, 28, 30, 31, 33, 34 e 35
```

Eles cobrem o núcleo com maior chance de cair.

38. Conexão Com A Próxima Aula

Nesta aula, você aprendeu a ler uma instrução por dentro:

```
opcode
registradores
imediato
endereço
modo de endereçamento
```

A próxima aula entra em:

```
datapath e controle
```

Conexão direta:

os campos da instrução controlam o datapath

Exemplo:

opcode ADD -> controle manda ULA somar
rd -> banco de registradores sabe onde escrever
rs1/rs2 -> banco de registradores sabe o que ler
LOAD -> controle ativa leitura da memória
STORE -> controle ativa escrita da memória
BEQ -> controle decide se PC muda

Ou seja:

Aula 26 = como a instrução é escrita/codificada
Aula 27 = como o hardware executa essa instrução